

TERRAFORM WORKSPACE DEFINITION

Terraform Workspace is a Terraform feature used to manage multiple infrastructure environments and separate state files using the same Terraform configuration. It also supports modular infrastructure management by organizing reusable code using Terraform modules.

Terraform Workspace Uses

- Manages multiple environments like staging, preprod, and production using the same Terraform configuration.
- Maintains separate state files for each workspace to isolate infrastructure resources.
- Simplifies environment switching and infrastructure management in DevOps workflows.

TERRAFORM MODULES DEFINITION

Terraform Modules are reusable and organized collections of Terraform configuration files used to simplify infrastructure management and avoid repeating the same code multiple times.

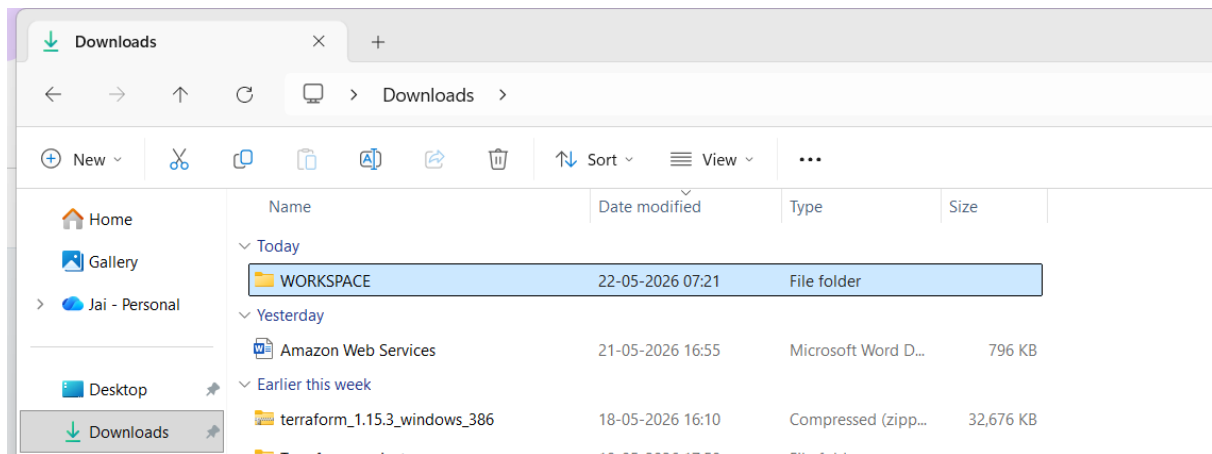
Terraform Modules Uses

- Reuses the same Terraform code across multiple projects and environments.
- Organizes infrastructure into smaller and manageable components.
- Reduces code duplication and simplifies infrastructure maintenance.

HANDS-ON SCREENSHOTS

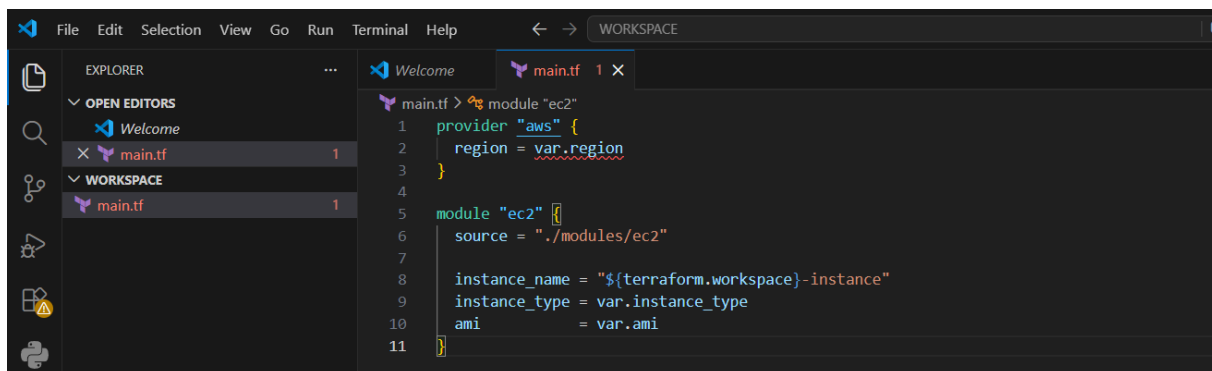
Screenshot: Create Terraform Workspace Project Folder

- Open Downloads folder in local system
- Create a new folder named workspace
- Terraform workspace project folder created successfully



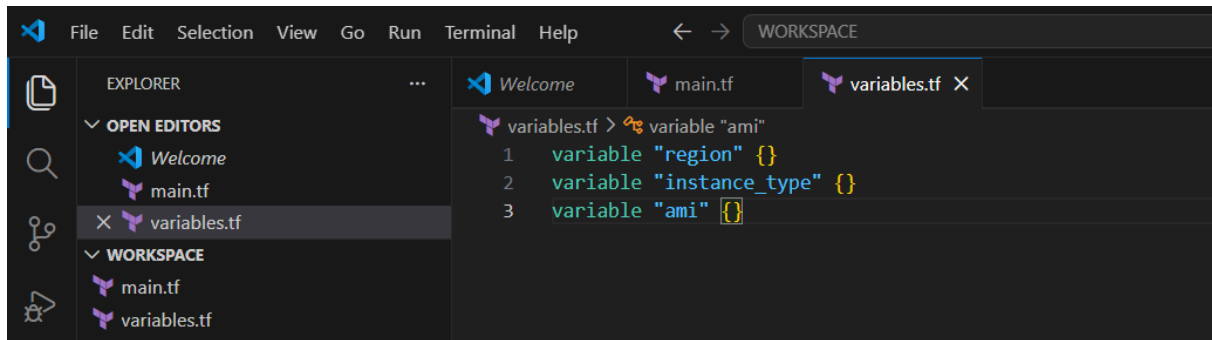
Screenshot: Create Root Module File

- Open Visual Studio Code
- Click File → Open Folder
- Select the created workspace folder and open it
- Create a new file named main.tf for the root module
- Paste the Terraform root module configuration code into main.tf
- Save the main.tf file successfully



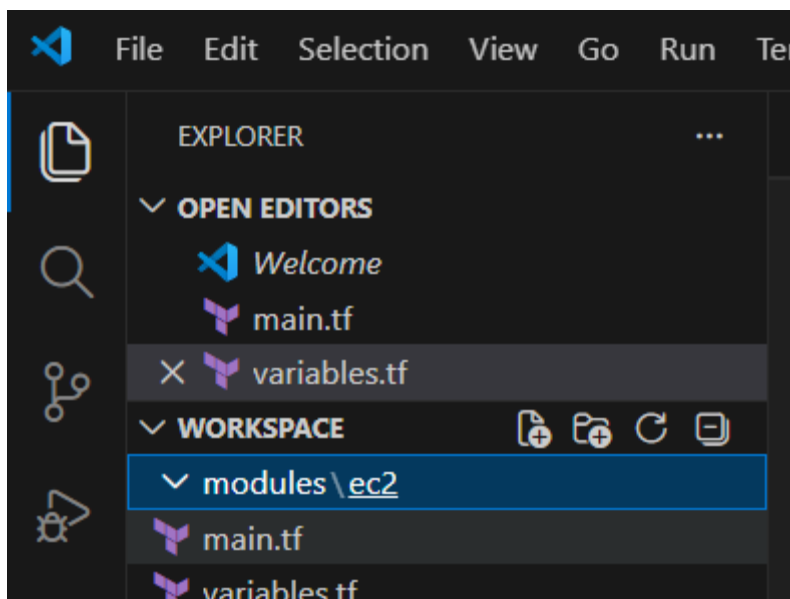
Screenshot: Create Variables File in Root Module

- Create a new file named variables.tf in the root module
- Paste the variable declaration code into variables.tf
- Save the variables.tf file successfully



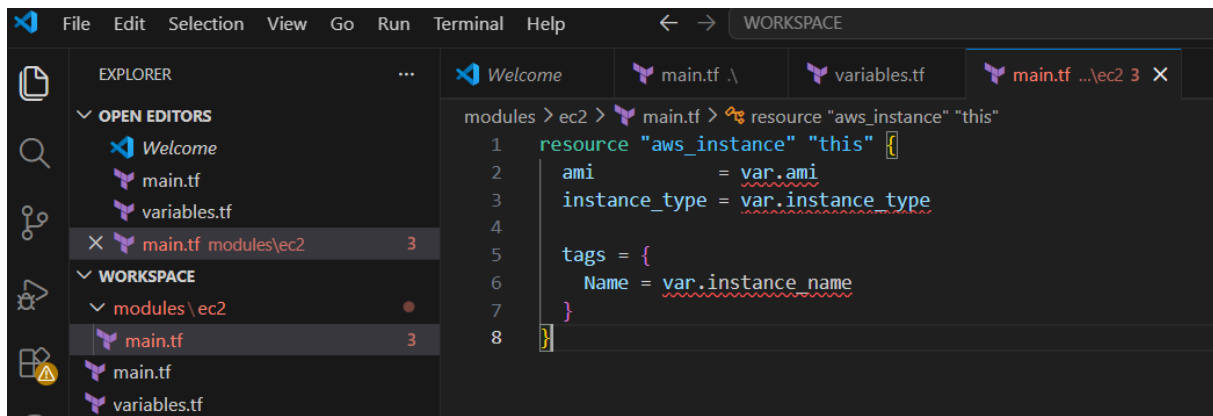
Screenshot: Create Terraform Module Folder Structure

- Create a new folder named modules inside the workspace folder
- Open the modules folder
- Create another folder named ec2 inside the modules folder
- Terraform module folder structure created successfully



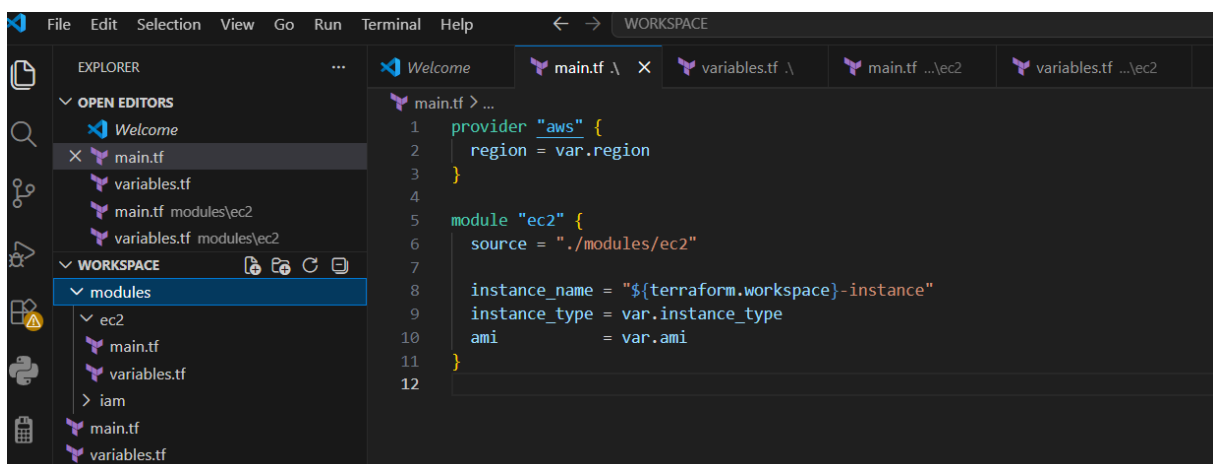
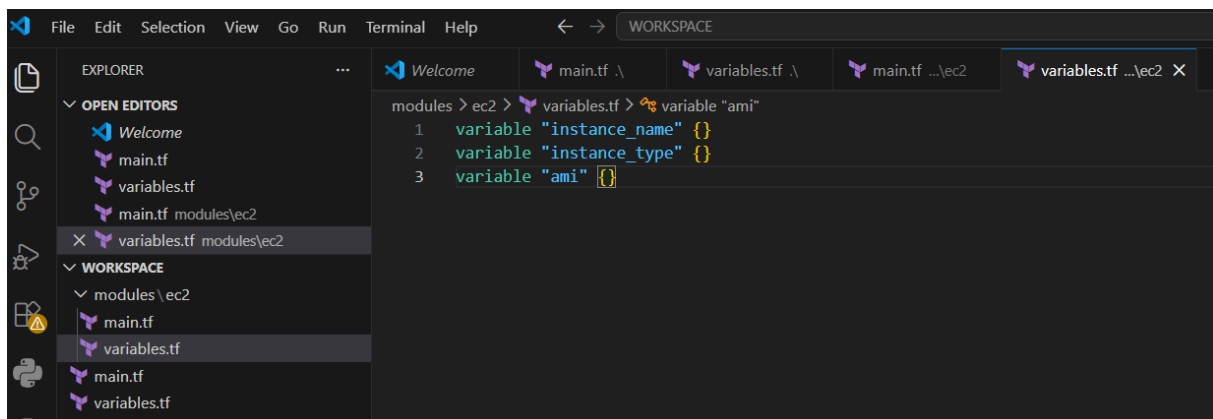
Screenshot: Add EC2 Module Main File

- Open the modules/ec2 folder
- Create a new file named main.tf inside the EC2 module
- Paste the EC2 resource configuration code into main.tf
- Save the main.tf file successfully



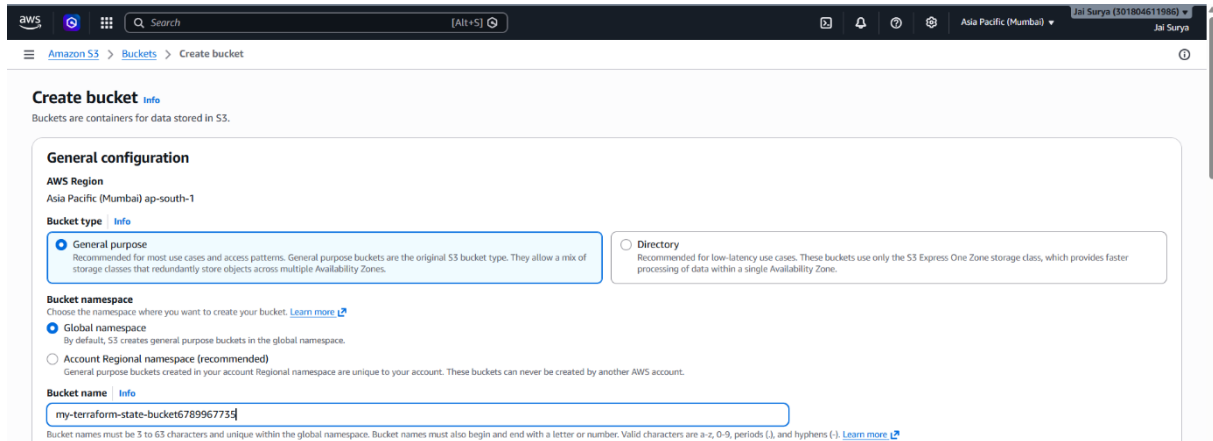
Screenshot: Create Variables File in EC2 Module

- Open the modules/ec2 folder
- Create a new file named variables.tf inside the EC2 module
- Paste the variable declaration code into variables.tf
- Save the variables.tf file successfully



Screenshot: Create S3 Bucket for Terraform State File

- Open AWS Management Console
- Go to S3 service
- Click Create bucket
- Enter a unique bucket name



Create bucket [Info](#)

Buckets are containers for data stored in S3.

General configuration

AWS Region
Asia Pacific (Mumbai) ap-south-1

Bucket type [Info](#)

☒ **General purpose**
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ **Directory**
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket namespace
Choose the namespace where you want to create your bucket. [Learn more](#)

☒ **Global namespace**
By default, S3 creates general purpose buckets in the global namespace.

☐ **Account Regional namespace (recommended)**
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

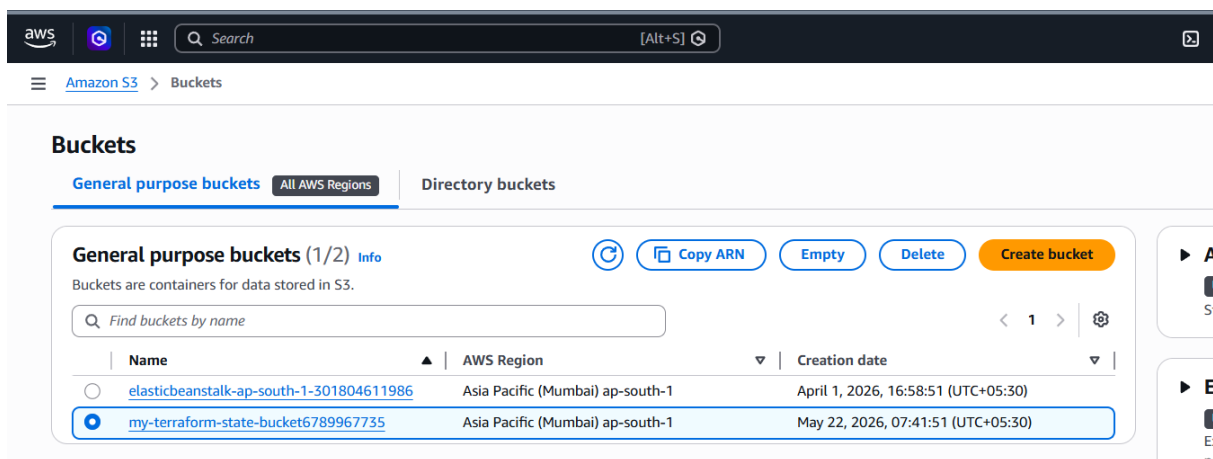
Bucket name [Info](#)

my-terraform-state-bucket6789967735

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

Screenshot: Verify S3 Bucket Creation

- Open AWS Management Console
- Go to S3 Dashboard
- Verify the newly created bucket in the bucket list
- S3 bucket created successfully for Terraform state file storage



Buckets

[General purpose buckets](#) [All AWS Regions](#) [Directory buckets](#)

General purpose buckets (1/2) [Info](#)

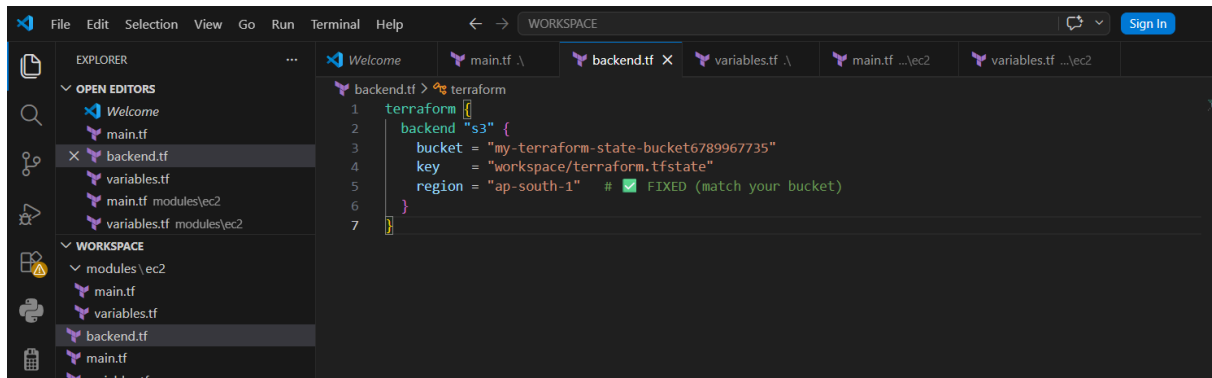
Buckets are containers for data stored in S3.

[Refresh](#) [Copy ARN](#) [Empty](#) [Delete](#) [Create bucket](#)

	Name	AWS Region	Creation date
☐	elasticbeanstalk-ap-south-1-301804611986	Asia Pacific (Mumbai) ap-south-1	April 1, 2026, 16:58:51 (UTC+05:30)
☒	my-terraform-state-bucket6789967735	Asia Pacific (Mumbai) ap-south-1	May 22, 2026, 07:41:51 (UTC+05:30)

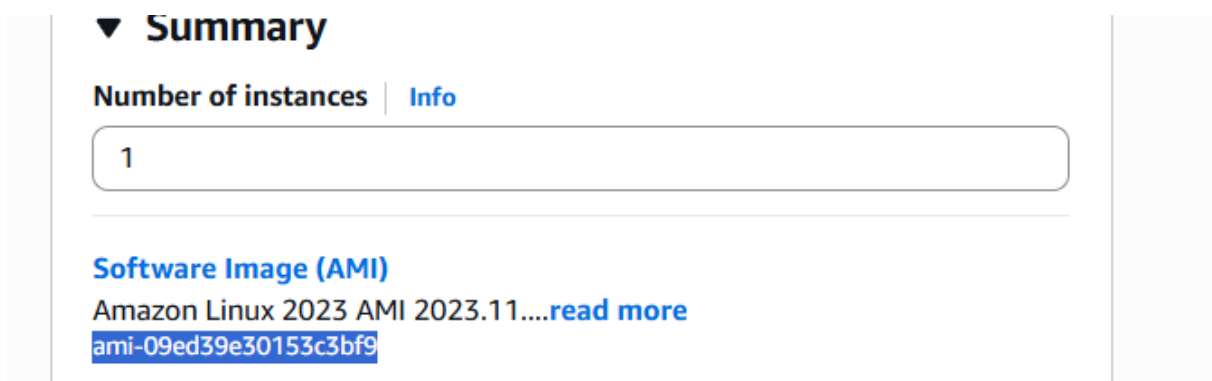
Screenshot: Create Backend Configuration File

- Create a new file named backend.tf in the root module
- Paste the Terraform backend configuration code into backend.tf
- Use the same S3 bucket name created in AWS
- Use the same AWS region where the S3 bucket was created
- Save the backend.tf file successfully



Screenshot: Copy AMI ID from EC2 Console

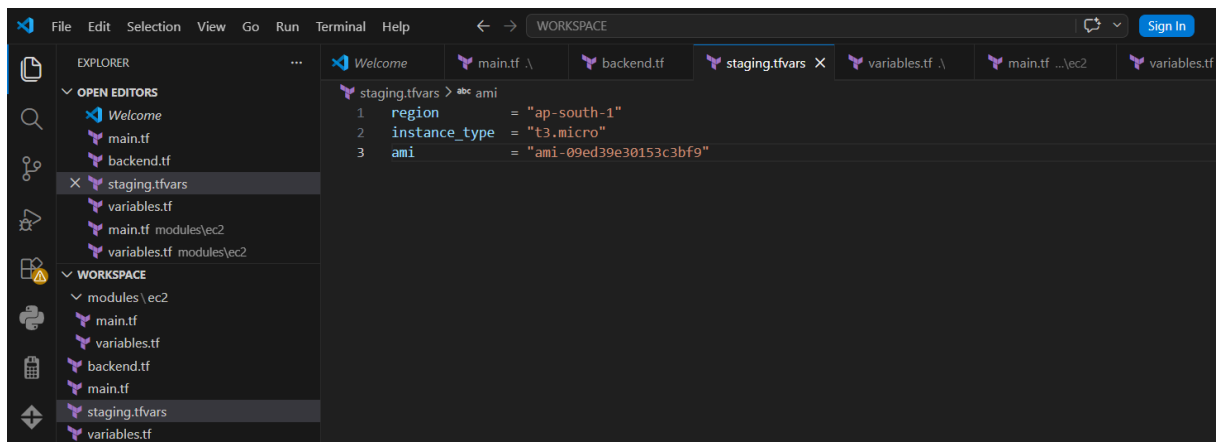
- Open AWS Management Console
- Go to EC2 Dashboard
- Open Launch Instance page or Instances section
- Copy the required AMI ID for Terraform configuration use



Screenshot: Create Staging Variable Values File

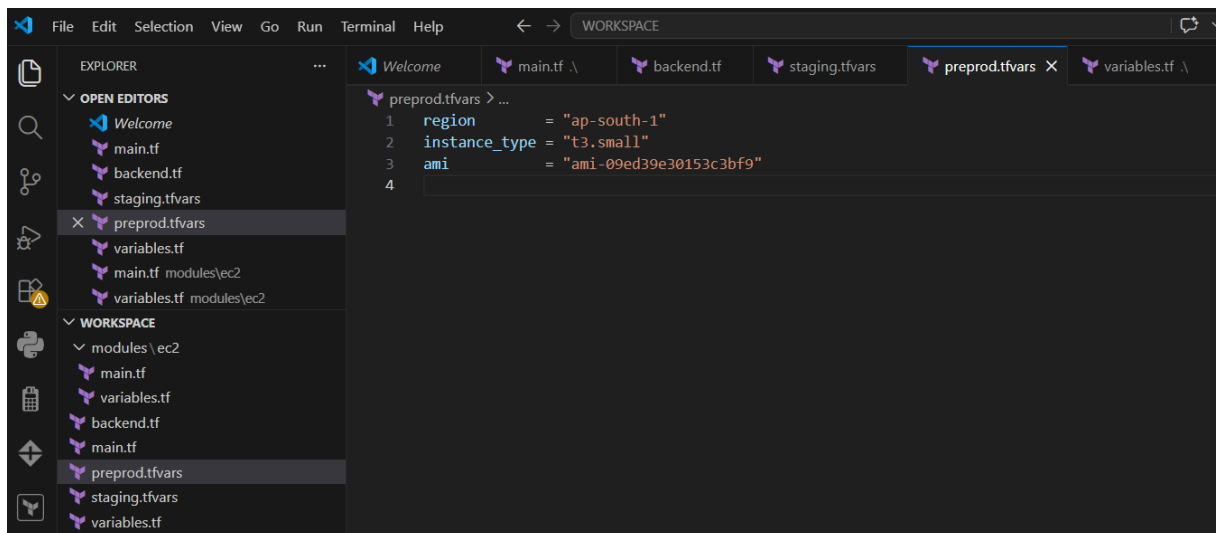
- Create a new file named staging.tfvars in the root module
- Paste the staging environment variable values into the file

- Add region, instance type, and AMI ID values
- Save the staging.tfvars file successfully



Screenshot: Create Preprod Variable Values File

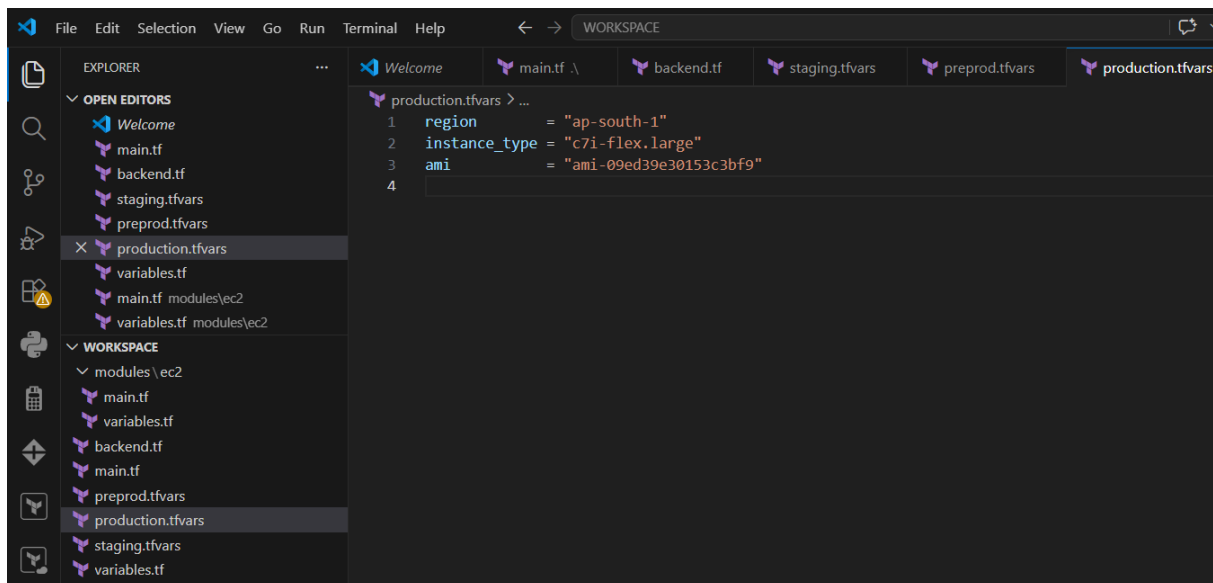
- Create a new file named preprod.tfvars in the root module
- Paste the preprod environment variable values into the file
- Set instance type as t3.small
- Add region and AMI ID values
- Save the preprod.tfvars file successfully



Screenshot: Create Production Variable Values File

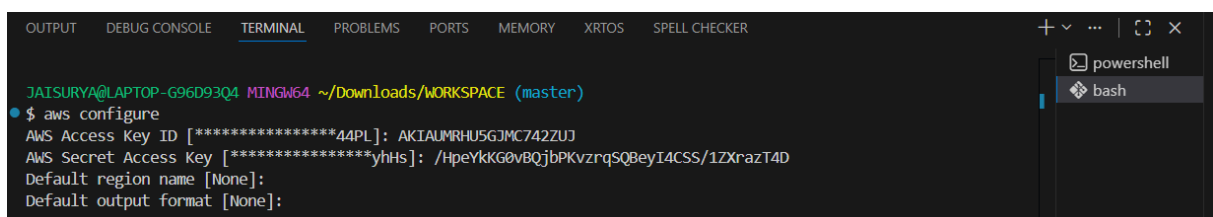
- Create a new file named production.tfvars in the root module
- Paste the production environment variable values into the file

- Add region, instance type, and AMI ID values
- Save the production.tfvars file successfully



Screenshot: Configure AWS CLI Credentials

- Open terminal in VS Code
- Execute aws configure
- Enter AWS Access Key ID
- Enter AWS Secret Access Key
- Enter default AWS region name
- AWS CLI configuration completed successfully



Screenshot: Initialize Terraform Workspace Project

- Execute terraform init
- Terraform initializes the working directory


```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
$ terraform init
Initializing modules...
- ec2 in modules\ec2
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
```

Screenshot: List Terraform Workspaces

- Execute terraform workspace list
- Terraform displays available workspaces
- Default workspace shown successfully as default

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
$ terraform workspace list
* default
```

Screenshot: Create Terraform Staging Workspace

- Execute terraform workspace new staging
- New workspace named staging created successfully
- Terraform switched to the staging workspace automatically

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
$ terraform workspace new staging
Created and switched to workspace "staging"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
```

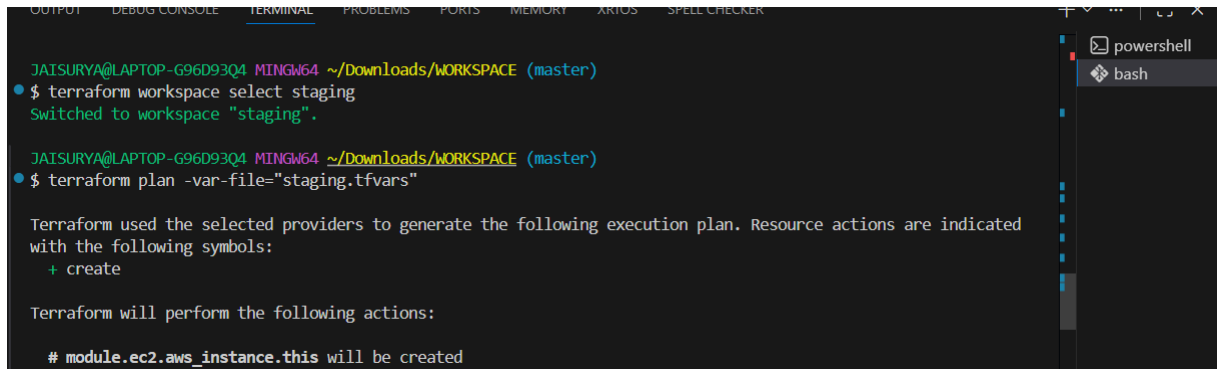
Screenshot: Create and List Terraform Workspaces

- Execute terraform workspace new preprod
- Execute terraform workspace new production
- Execute terraform workspace list
- Terraform displays all available workspaces: default, staging, preprod, and production

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
$ terraform workspace list
default
preprod
* production
staging
```

Screenshot: Execute Terraform Plan for Staging Environment

- Execute terraform workspace select staging
- Terraform switched successfully to the staging workspace
- Execute terraform plan -var-file=staging.tfvars
- Terraform loads variable values from staging.tfvars
- Execution plan generated successfully for the staging workspace infrastructure



```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
• $ terraform workspace select staging
Switched to workspace "staging".

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
• $ terraform plan -var-file="staging.tfvars"

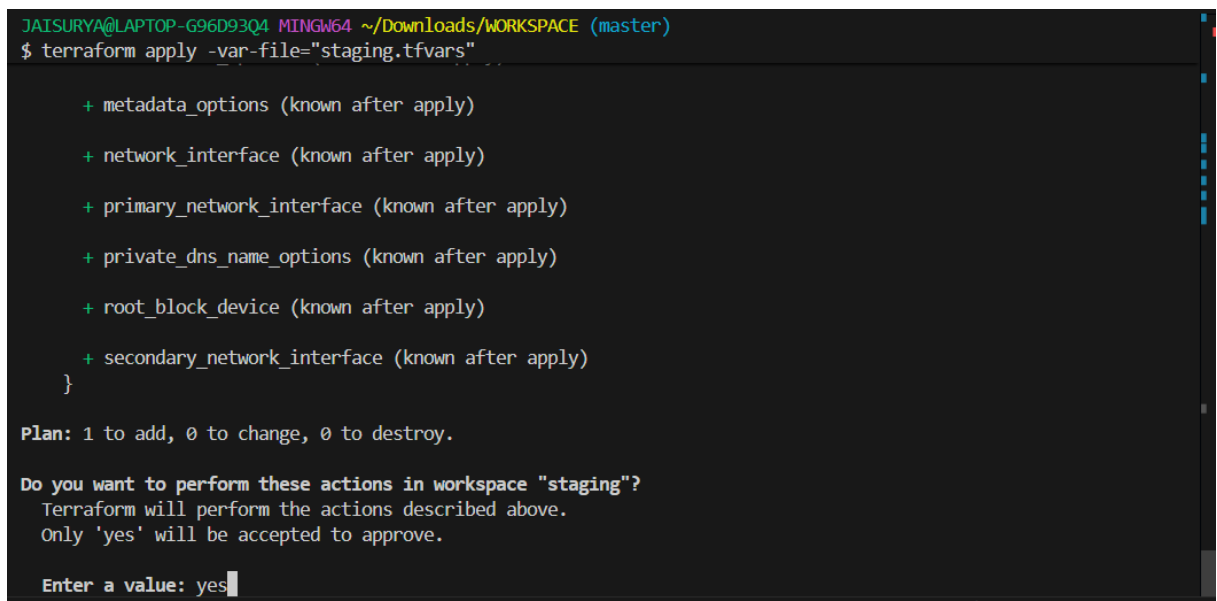
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  + create

Terraform will perform the following actions:

# module.ec2.aws_instance.this will be created
```

Screenshot: Apply Terraform Configuration for Staging Workspace

- Execute terraform apply -var-file=staging.tfvars
- Terraform displays infrastructure creation details
- Type yes to approve resource creation
- Staging environment resources created successfully using Terraform workspace



```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
$ terraform apply -var-file="staging.tfvars"

+ metadata_options (known after apply)
+ network_interface (known after apply)
+ primary_network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
+ secondary_network_interface (known after apply)
}

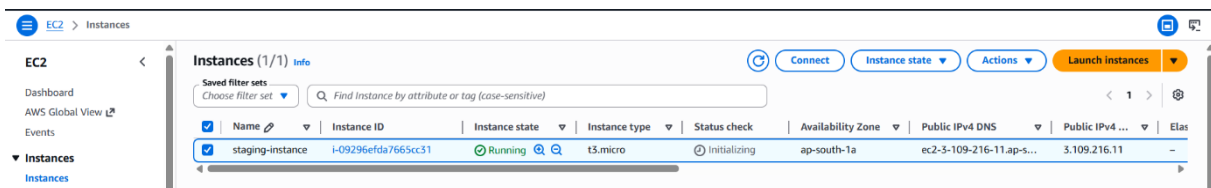
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions in workspace "staging"?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes
```

Screenshot: Verify Staging EC2 Instance in AWS Console

- Open AWS Management Console
- Go to EC2 Dashboard
- Open Instances section
- Verify the EC2 instance created from the staging workspace
- Staging instance created successfully using Terraform workspace



Screenshot: Deploy Preprod Environment Using Terraform Workspace

- Execute terraform workspace select preprod
- Terraform switched successfully to the preprod workspace
- Execute terraform plan -var-file=preprod.tfvars
- Terraform displays execution plan for preprod environment
- Execute terraform apply -var-file=preprod.tfvars
- Type yes to approve infrastructure creation
- Preprod environment resources created successfully using Terraform workspace

```
OUTPUT  DEBUG CONSOLE  TERMINAL  PROBLEMS  PORTS  MEMORY  XRTOS  SPELL CHECKER

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
• $ terraform workspace select preprod
Switched to workspace "preprod".

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
• $ terraform plan -var-file="preprod.tfvars"

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  + create

Terraform will perform the following actions:

# module.ec2.aws_instance.this will be created
+ resource "aws_instance" "this" {
```

```
JATISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
$ terraform apply -var-file="preprod.tfvars"

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  + create

Terraform will perform the following actions:
```

Screenshot: Deploy Production Environment Using Terraform Workspace

- Execute terraform workspace select production
- Terraform switched successfully to the production workspace
- Execute terraform plan -var-file=production.tfvars
- Terraform displays execution plan for production environment
- Execute terraform apply -var-file=production.tfvars
- Type yes to approve infrastructure creation
- Production environment resources created successfully using Terraform workspace

```
OUTPUT  DEBUG CONSOLE  TERMINAL  PROBLEMS  PORTS  MEMORY  XRTOS  SPELL CHECKER

JATISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
$ terraform workspace select production
Switched to workspace "production".

JATISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
$ terraform plan -var-file="production.tfvars"

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  + create

Terraform will perform the following actions:

# module.ec2.aws_instance.this will be created
+ resource "aws_instance" "this" {
```

```
JATISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
$ terraform apply -var-file="production.tfvars"

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  + create

Terraform will perform the following actions:
```

Screenshot: Verify All Workspace Instances in AWS Console

- Open AWS Management Console
- Go to EC2 Dashboard
- Open Instances section

- Verify instances created from staging, preprod, and production workspaces
- All three EC2 instances created successfully using Terraform workspaces

Instances (3/3) info

Save filter sets: Choose filter set, Find Instance by attribute or tag (case-sensitive)

☒	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elas
☒	production-ins...	i-050ca31bbe27d7a1b	Running	c7i-flex.large	Initializing	ap-south-1b	ec2-35-154-1-51.ap-so...	35.154.1.51	-
☒	staging-instance	i-09296efda7665cc31	Running	t3.micro	3/3 checks passsec	ap-south-1a	ec2-3-109-216-11.ap-s...	3.109.216.11	-
☒	preprod-insta...	i-047820303b29b41e2	Running	t3.small	3/3 checks passsec	ap-south-1a	ec2-13-126-33-126.ap-...	13.126.33.126	-

Screenshot: Verify Terraform State Files in S3 Bucket

- Open AWS Management Console
- Go to S3 service
- Open the Terraform state file bucket
- Open the env/ folder inside the bucket
- Verify preprod, production, and staging workspace folders
- Check terraform.tfstate files stored for each workspace successfully

Amazon S3 > Buckets > my-terraform-state-bucket6789967735

my-terraform-state-bucket6789967735 info

Objects (1)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	env/	Folder	-	-	-

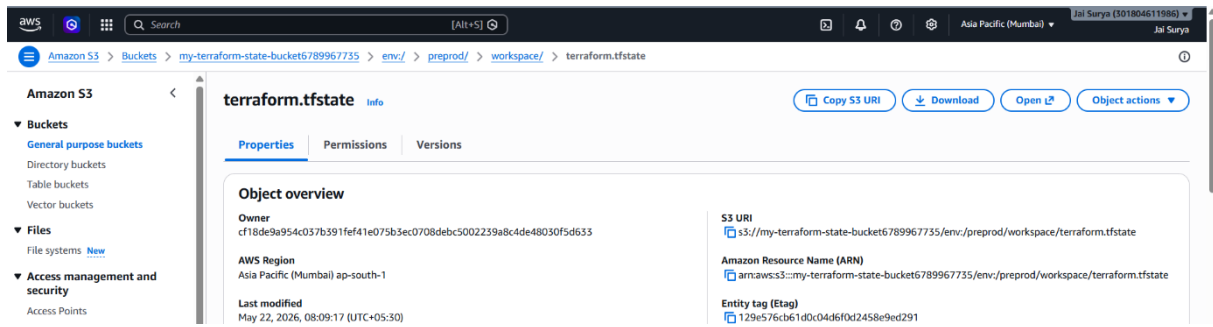
Amazon S3 > Buckets > my-terraform-state-bucket6789967735 > env/

env:/

Objects (3)

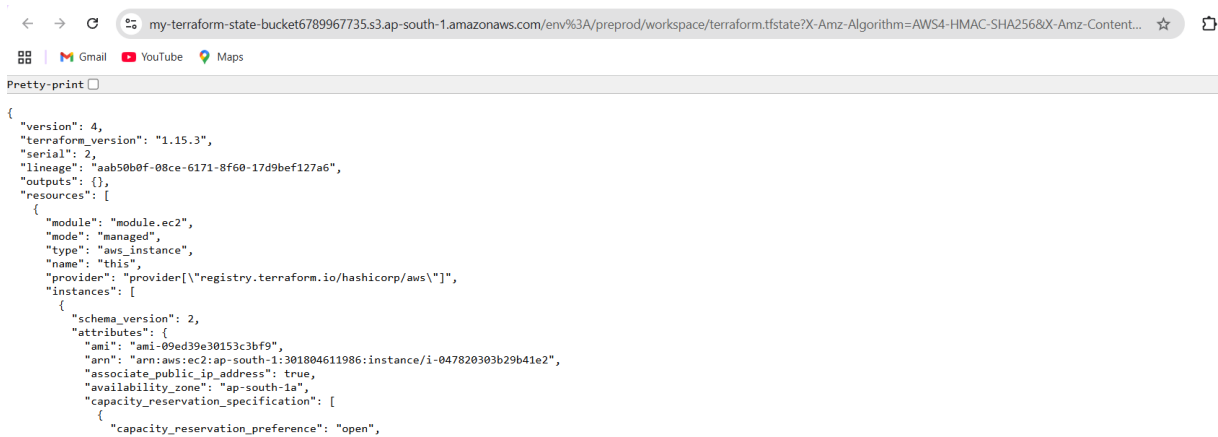
Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	preprod/	Folder	-	-	-
☐	production/	Folder	-	-	-
☐	staging/	Folder	-	-	-



Screenshot: Open Terraform State File in S3 Bucket

- Open the required workspace folder inside the env/ directory
- Open the terraform.tfstate object
- Terraform state file contents displayed successfully for the selected workspace



Screenshot: Destroy Terraform Workspace Resources

- Execute terraform destroy -var-file=staging.tfvars for staging workspace
- Execute terraform destroy -var-file=preprod.tfvars for preprod workspace
- Execute terraform destroy -var-file=production.tfvars for production workspace
- Type yes to approve infrastructure deletion
- All workspace resources deleted successfully using Terraform workspaces

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/WORKSPACE (master)
$ terraform destroy -var-file="production.tfvars"
module.ec2.aws_instance.this: Refreshing state... [id=i-050ca31bbe27d7a1b]

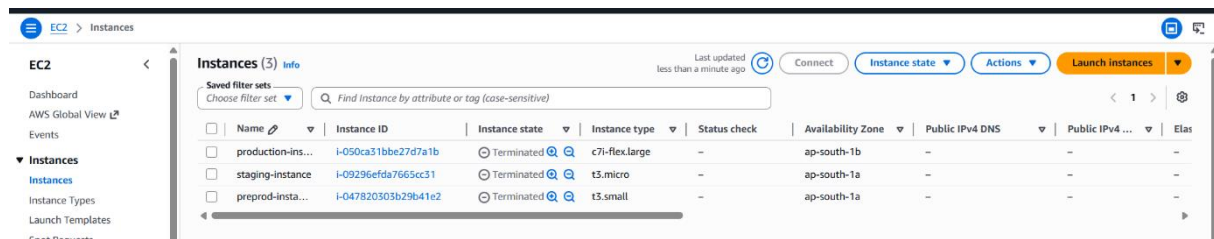
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  - destroy

Terraform will perform the following actions:

# module.ec2.aws_instance.this will be destroyed
- resource "aws_instance" "this" {
```

Screenshot: Verify EC2 Instance Termination

- Open AWS Management Console
- Go to EC2 Dashboard
- Open Instances section
- Verify staging, preprod, and production instances are terminated successfully using Terraform destroy



The screenshot shows the AWS Management Console for the EC2 service, specifically the 'Instances' page. The page displays a list of three EC2 instances, all of which are in the 'Terminated' state. The instances are named 'production-ins', 'staging-instance', and 'preprod-insta'. The table includes columns for Name, Instance ID, Instance state, Instance type, Status check, Availability Zone, Public IPv4 DNS, Public IPv4 ..., and Elas.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elas
production-ins...	i-050ca31bbe27d7a1b	Terminated	c7i-flex.large	-	ap-south-1b	-	-	-
staging-instance	i-09296efda7665cc31	Terminated	t3.micro	-	ap-south-1a	-	-	-
preprod-insta...	i-047820303b29b41e2	Terminated	t3.small	-	ap-south-1a	-	-	-